

## Week 03

### 1) Evaluating a Model

- \* When we build a ML model, we want it to learn patterns from data, not just memorize them.
- \* To know if it's learning well, we must test it on data it has never seen before.

### Training Set

- \* The part of the dataset used to train the model (adjust the model's parameters so it fits the data. (70%, 80%...))

### Test set

- \* A separate portion usually the remaining 20% used to <sup>evaluate</sup> how well the trained model performs on unseen data.

### Train Error vs Test Error

- ▢ Train Error  $\rightarrow$  The error (or loss) the model makes on the training data
- ▢ Test Error  $\rightarrow$  The error (or loss) the model makes on the test data
- \* Goal is to achieve low test error not just low train error



## Train / Test procedure for linear regression

### Squared error cost

- \* Fit parameters by minimizing the cost function

$$J(\vec{w}, b) = \left[ \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m_{\text{train}}} \sum_{j=1}^n w_j^2 \right]$$

- \* Compute test error

$$J_{\text{test}}(\vec{w}, b) = \frac{1}{2m_{\text{test}}} \left[ \sum_{i=1}^{m_{\text{test}}} (f_{\vec{w}, b}(\vec{x}_{\text{test}}^{(i)}) - y_{\text{test}}^{(i)})^2 \right]$$

- \* Compute train error

$$J_{\text{train}}(\vec{w}, b) = \frac{1}{2m_{\text{train}}} \left[ \sum_{i=1}^{m_{\text{train}}} (f_{\vec{w}, b}(\vec{x}_{\text{train}}^{(i)}) - y_{\text{train}}^{(i)})^2 \right]$$

## Train / Test procedure for Classification

- \* Similarly as above calculate  $(J_{\text{test}}(\vec{w}, b) / J_{\text{train}}(\vec{w}, b))$  errors.  
(log loss)

- \*  $J_{\text{test}}(\vec{w}, b)$  is the fraction of the test set that has been misclassified



\*  $J_{\text{train}}(\vec{w}, b)$  is the fraction of the train set that has been misclassified.

$$\hat{y} = \begin{cases} 1 & \text{if } f_{\vec{w}, b}(\vec{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } f_{\vec{w}, b}(\vec{x}^{(i)}) < 0.5 \end{cases}$$

Count  $\hat{y} \neq y$

02) Cross validation set & Model Selection

Cross validation set

\* The model does not train on this data

\* But we check the error on the CV dataset and it will help to compare multiple models and choose the best one

\* Used for hyperparameter tuning (like learning rate, number of layers, etc.)

Model Selection

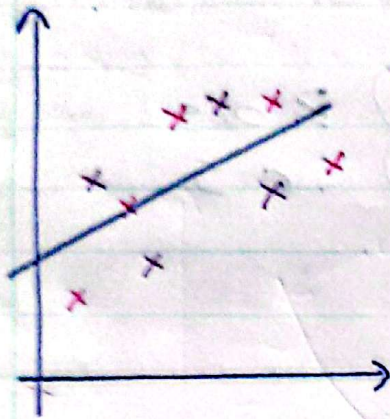
\* Train the models on training dataset

\* Compute the CV error for all models (using  $k$ -fold cross validation) on CV dataset

\* Choose the model with lowest CV error and calculate the test error on that model (generalization error)

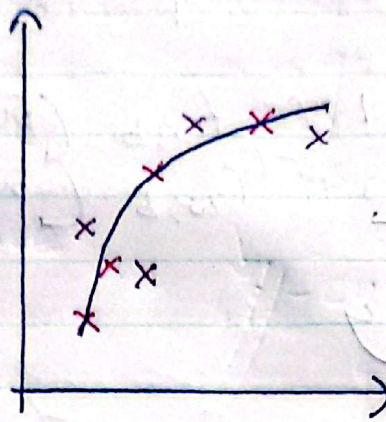


### 03) Diagnosing Bias & Variance



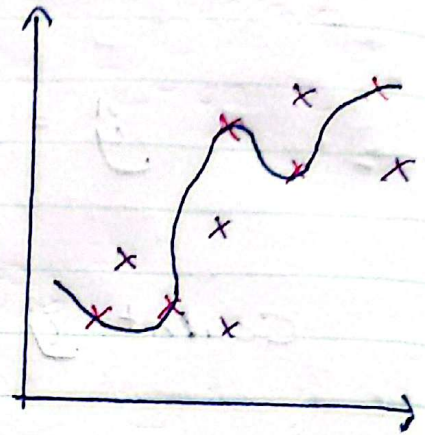
$d=1$   
 $J_{\text{train}}$  is high  
 $J_{\text{cv}}$  is high

High bias  
 (underfit)



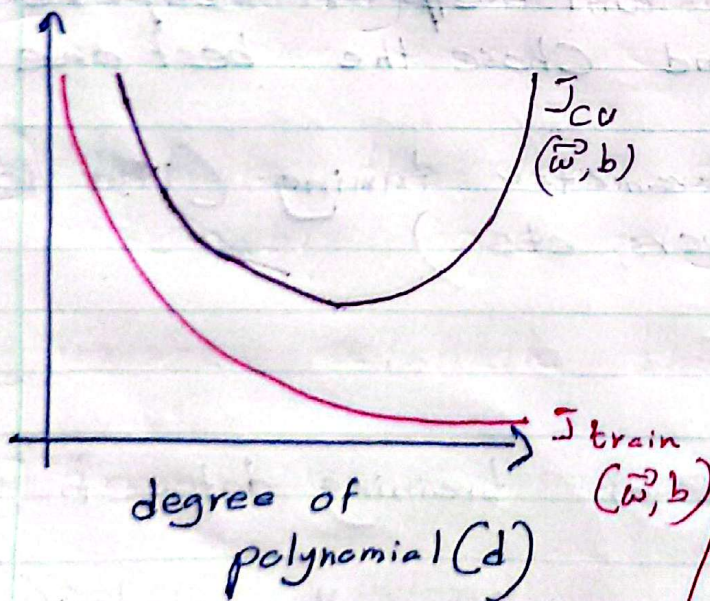
$d=2$   
 $J_{\text{train}}$  is low  
 $J_{\text{cv}}$  is low

Just right



$d=3$   
 $J_{\text{train}}$  is low  
 $J_{\text{cv}}$  is ~~low~~ high

High variance  
 (overfit)



1) High bias (underfit)

\*  $J_{\text{train}}$  will be high  
 ( $J_{\text{train}} \approx J_{\text{cv}}$ )

2) High Variance (overfit)

\*  $J_{\text{cv}} \gg J_{\text{train}}$   
 ( $J_{\text{train}}$  may be low)

3) High bias & High Variance

\*  $J_{\text{train}}$  will be high  
 ( $J_{\text{cv}} \gg J_{\text{train}}$ )



#### 04) Regularization and Bias / Variance

Linear Regression with regularization

$$\text{Model: } f_{\vec{w}, b}(x) = w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + b$$

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

$$\left. \begin{array}{l} 1) \text{ Large } \lambda \\ (\lambda = 10,000) \rightarrow w_1 \approx 0, w_2 \approx 0 \\ f(\vec{w}, b)(\vec{x}) \approx b \end{array} \right\} J_{\text{train}}(\vec{w}, b) \text{ is large}$$

\* High bias (Underfit)

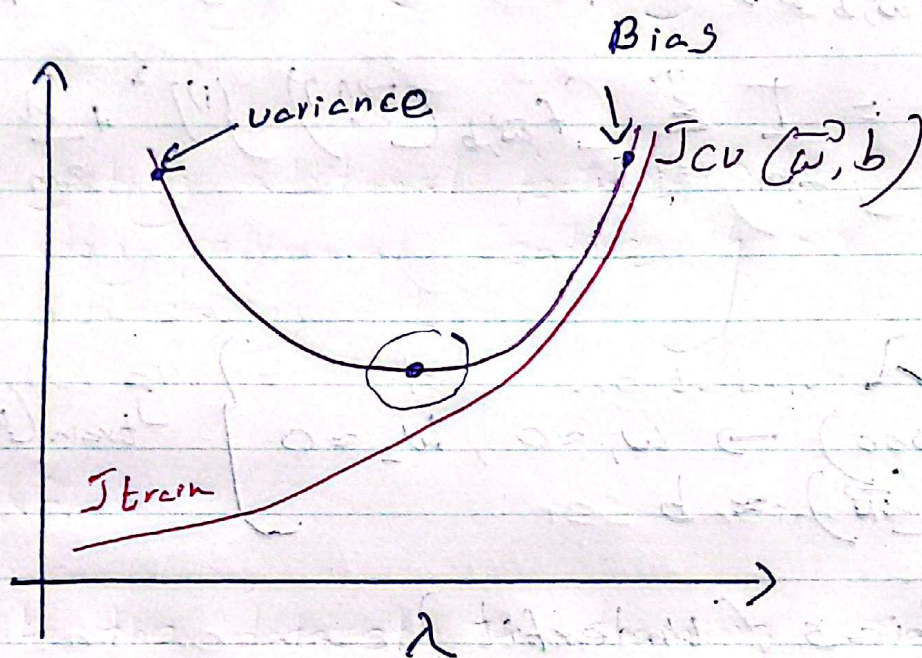
$$\left. \begin{array}{l} 2) \text{ Intermediate } \lambda \\ * \text{ Just right } \end{array} \right\} \begin{array}{l} * J_{\text{train}}(\vec{w}, b) \text{ is small} \\ * J_{\text{CV}}(\vec{w}, b) \text{ is small} \end{array}$$

$$\left. \begin{array}{l} 3) \text{ Small } \lambda \\ (\lambda = 0) \\ * \text{ High Variance (overfit)} \end{array} \right\} \begin{array}{l} * J_{\text{train}}(\vec{w}, b) \text{ is small} \\ * J_{\text{CV}}(\vec{w}, b) \text{ is large} \end{array}$$

\* When choosing the regularization parameter ( $\lambda$ ) we centre multiple  $\lambda$  and evaluate  $J_{\text{CV}}$  for each. We pick the  $\lambda$  where  $J_{\text{CV}}$  is lowest. (train with cross validation dataset)



\* Then we pick the parameters of  $J_{cv}$  lowest  $(\vec{a}^*, b^*)$  and Report test error based on those parameters.  $(J_{test}(\vec{a}^*, b^*))$



35) Establishing a baseline level of performance

\* What is the level of error you can reasonably hope to get to?

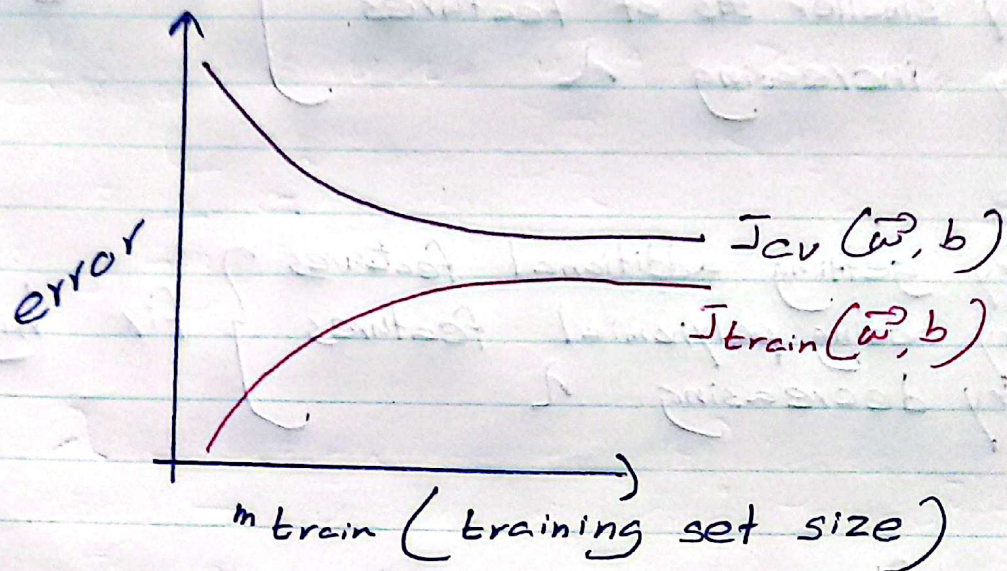
We can establish this by:

- 1) Human level performance
- 2) Computing algorithms performance  
(previously trained base models)
- 3) Guess based on experience

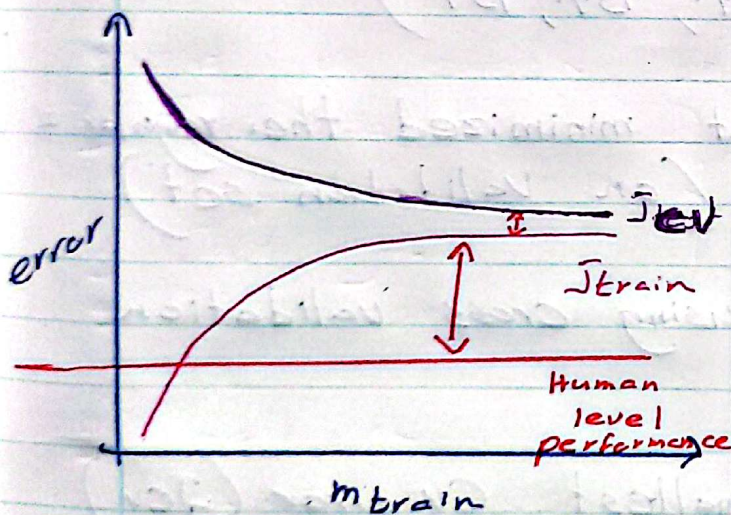


## 06) Learning Curves

$$f_{\vec{w}, b}(x) = w_1 x + w_2 x^2 + b$$

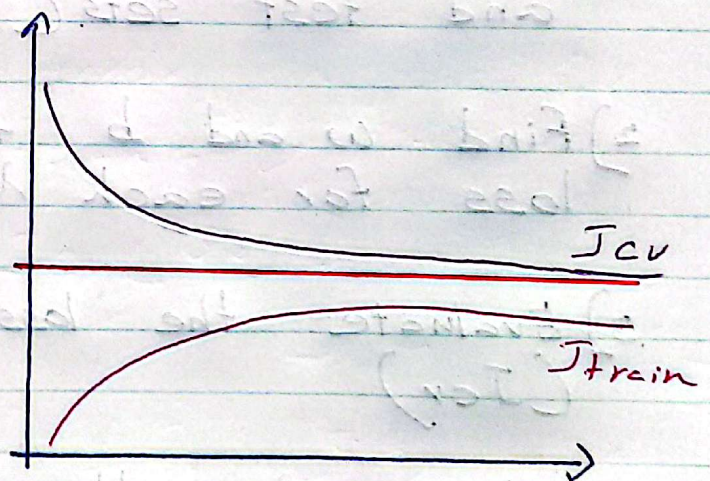


High bias



\* If a learning algorithm is high bias, getting more data will not (by itself) help much

High Variance



\* If a learning algorithm is high variance, getting more training data is likely to help



## 07) Debugging a learning algorithm

- \* ) Get more training examples
  - \* ) Try smaller sets of features
  - \* ) Try increasing  $\lambda$
- } fix high variance

- \* ) Try getting additional features
  - \* ) Try adding polynomial features
  - \* ) Try decreasing  $\lambda$
- } fix high bias

### Workflow

- 1) Split the dataset to training, validation and test sets (70%, 15%, 15%)
- 2) Find  $w$  and  $b$  that minimized the regularized loss for each  $\lambda$  (on validation set)
- 3) Evaluate the loss using cross validation ( $J_{cv}$ )
- 4) Chose  $\lambda$  with smallest CV error ( $J_{cv}$ )  
~~Choose~~
- 5) Retrain the model with all the training dataset with chosen  $\lambda$  to get final ( $w^*, b^*$ )
- 6) Evaluate on test ~~data~~ set with ( $w^*, b^*$ ) to report test error as estimated generalized error.



## 08] Error Analysis

- \* Error Analysis is about understanding model weaknesses before randomly change things
- \* we
  - i) Collect a sample of incorrect predictions
  - ii) Categorize Errors
  - iii) Measure error type frequency
- \* Then decide what to fix first based on which error categories dominate

## 09] Add Data

- \* When you identify gaps in your data, you can expand or enhance it.
- i) Data Augmentation (Natural)
  - \* You modify existing samples to create more training examples without changing meaning
- - Rotate, flip, crop or adjust brightness (images)
  - Add noise (for audio)
  - Synonym replacement (for text)
- \* Helps improve robustness and reduce overfitting



## 2) Data Synthesis (Artificial)

- \* You generate new examples that don't exist using algorithms or generative models.
- - Use GANs or diffusion model for synthetic images
  - Use LLMs to generate text samples
  - Use simulators for physics or robotic data
- \* This fills in missing distributions or rare cases. (Rare diseases)

## 10) Data Engineering Approach

### • Model-Centric Approach

- \* Historically most effort went into improving the model, not the data.
- \* If data is poor, even the best model fails.

### Data-Centric Approach

- \* It emphasizes improving data quality rather than just changing the model.
- \* A small model trained on high-quality, diverse, clean data often outperforms complex model on messy trained data.



## 11) Transfer learning

- \* Transfer learning is when we reuse knowledge learned from one task (usually with a large dataset) to improve performance on another, related but smaller task (small dataset)

### 1) Pretraining Phase

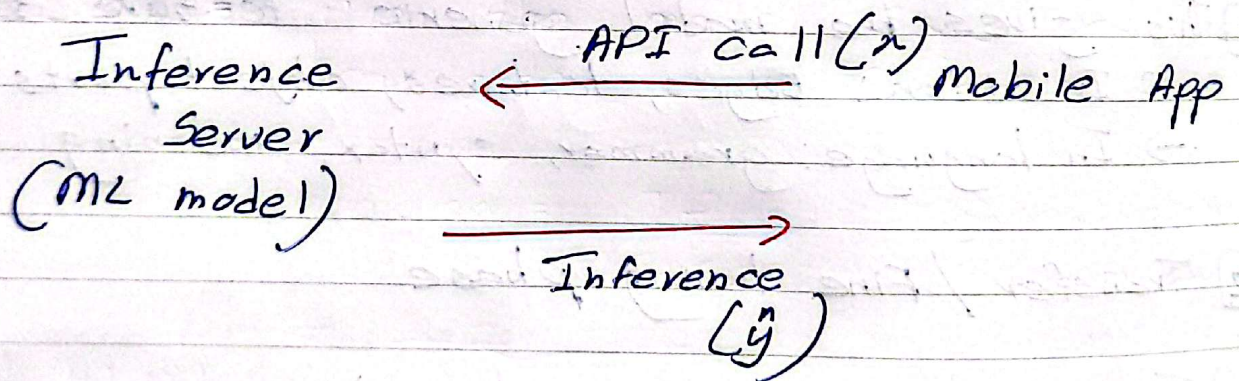
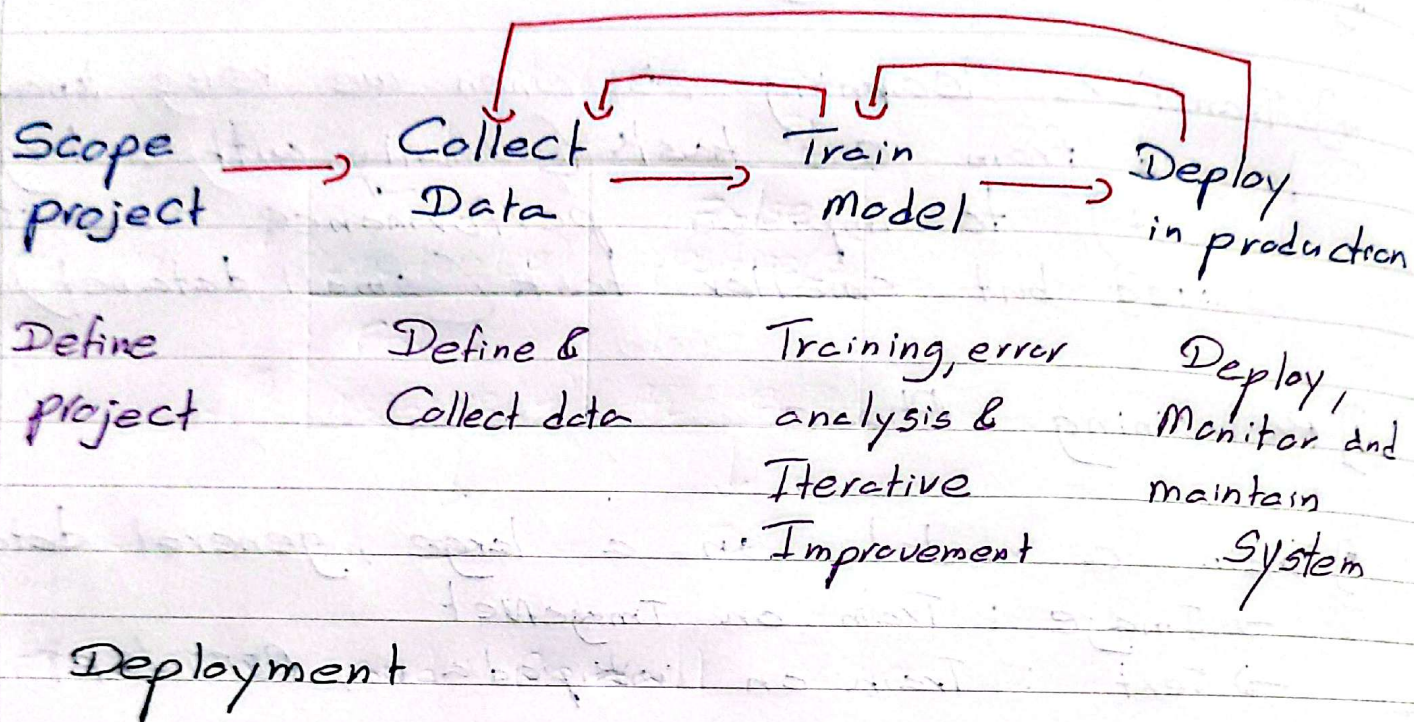
- Train a model on a large, general dataset.
  - Image : Train on ImageNet
  - Text : Train on wikipedia or, BERT/GPT
- This gives the model generic feature detectors
  - In vision: Edges, textures, object parts
  - In language: grammar, syntax, meaning

### 2) Transfer / Fine tuning phase

- Take the pretrain model and:
  - \* Freeze some layers (keep general features)
  - \* Replace final layers for the new task
  - \* Train only on the top layers or the whole model lightly on your small dataset
- \* When you have →
  - \* small or limited data
  - \* new task is related to the source task
  - \* You need faster training or better performance quickly



## 12) Full Cycle of ML project



\* SE may be needed for  $\Rightarrow$

- Ensure reliable & efficient predictions
  - Scaling
  - Logging
  - System monitoring
  - Model updates
- } MLOps



### 13) Error Metrics for Skewed data

#### Accuracy

\* Accuracy means how often your model is correct overall

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total predictions}}$$

Why accuracy fails (skewed data / Imbalanced data)

\* Accuracy works well when classes are balanced but fails badly when one class dominates

Ex: (skewed data)

• 995 healthy (negative)

• 5 people have cancer (positive)

Model that predicts 'no cancer' for everyone gets:

$$\text{Accuracy} = \frac{995}{1000} = 99.5\%$$

\* looks great, but it misses all 5 cancer cases



## Confusion Matrix

|                 |   | Actual class       |                    |
|-----------------|---|--------------------|--------------------|
|                 |   | 1                  | 0                  |
| Predicted class | 1 | True positive (15) | False positive (5) |
|                 | 0 | False negative (5) | True negative (70) |
|                 |   | ↓<br>25            | ↓<br>75            |

## Precision and Recall

Precision  $\Rightarrow$  When the model says positive, how often is it correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

$\Rightarrow$  measure the quality of positive predictions

where

where

$\Rightarrow$  High precision = few false alarms



Recall  $\Rightarrow$  Sensitivity or True positive Rate

$$\text{Recall} = \frac{TP}{TP + FN}$$

\* measures coverage of actual positives detected

\* High recall = few misses

Precision - Recall tradeoff

\* If you make the model more strict (predict positive only when very sure)

Precision  $\uparrow$ , Recall  $\downarrow$

\* If you make the model more lenient (predict positive easily)

precision  $\downarrow$ , Recall  $\uparrow$

Example :

In logistic regression or neural net we can change the probability threshold :





- \* Default  $\Rightarrow 0.5$
- \* Lower it to 0.3  $\rightarrow$  more positives detected (Recall  $\uparrow$ , Precision  $\downarrow$ )
- \* Raise it to 0.8  $\rightarrow$  fewer positives detected (Precision  $\uparrow$ , Recall  $\downarrow$ )

Medical diagnosis  $\Rightarrow$  high recall  
(catch all cases)

Spam filter  $\Rightarrow$  high precision  
(don't mark real emails as spam)

### F1 Score

- \* F1 score combines precision and recall into a single number using harmonic mean

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- \* F1 becomes high when P & R both are good.

Balanced dataset  $\rightarrow$  Accuracy  
Imbalanced  $\rightarrow$  Precision, Recall, F1  
Minimize FN  $\rightarrow$  Recall  
Minimize FP  $\rightarrow$  Precision  
Balance both  $\rightarrow$  F1 score